

第 7 章

例外和中断的支持

在前面的几章中，我们已经实现了一个支持 56 条指令的 MIPS 流水线 CPU。不过，到目前为止，我们实现的都是指令系统的用户态部分。为了最终可以基于所实现的 CPU 搭建出一个小型的计算机系统，我们还要逐步添加指令系统中的特权态部分。这一章我们先介绍如何在 CPU 中支持例外（Exception）和中断（Interrupt）功能。

【本章学习目标】

- 理解例外和中断的软硬件协调处理机制。
- 理解精确例外的概念和处理方法。
- 掌握在流水线 CPU 中添加例外和中断支持的方法。

【本章实践目标】

本章最后有两个实践任务，请读者在学习完本章内容后，依次完成这些实践任务。

7.1 例外和中断的基本概念

有关例外和中断的基本概念，读者可以参考《计算机体系结构基础（第 2 版）》的 3.2 节或其他资料学习。这里我们将根据 CPU 设计的需要，梳理其中关键的概念。从 CPU 实现的角度来看，中断也是一种特殊的例外，所以在下文的表述中，除非专指中断，否则我们将统一用“例外”一词代指“例外”和“中断”。

7.1.1 例外是一套软硬件协同处理的机制

首先我们必须明确，例外是一套软硬件协同处理的机制。顾名思义，“例外”不是常态。例外对应的情况发生的频度不高，但处理起来比较复杂。本着“好钢用在刀刃上”的设计原则，我们希望尽可能由软件程序而不是硬件逻辑来处理这些复杂的异常情况。这样做既能保证硬件的设计复杂度得到控制，又能确保系统的实际运行性能没有太大的损失。

例外处理的绝大多数工作是由例外处理程序（软件）完成的，但是例外处理的开始和结

束阶段必须要由硬件来完成。

1. 开始阶段

例外触发条件的判断由硬件完成，例外的类型、触发例外的指令的 PC 等供例外处理程序使用的信息由硬件自动保存。此外，硬件需要跳转到例外处理程序的入口执行，并保证跳转到例外入口后，处理器处于高特权等级。随后，例外处理程序接管后续处理过程。

2. 结束阶段

例外处理程序完成所有处理之后，需要返回发生例外的指令（或者一个事前指定的程序入口）处重新开始执行，这个过程除了要执行一次跳转外，还需要将处理器的特权等级调整回最初发生例外的指令所处运行环境的特权等级。

7.1.2 精确例外

为了在 CPU 中正确地实现例外功能，我们有必要强调一下“精确例外”这个概念。什么叫精确例外呢？它要达到的效果是：当系统软件处理完例外返回后，对于发生例外的指令和它后面（程序序）的指令，就好像例外没有发生过一样。

在上面这个表述中，是让谁觉得“像没有发生过例外一样”？是指令，所以说，内容没有发生变化是从指令的视角来看的。这里我们用“看”这个拟人的修辞方式来表述，虽然生动，但从概念的角度难免不够精准。我们还需要再啰嗦几句。

指令能“看”到的内容有什么？它只能看到 ISA 中定义的那些内容，看不到那些不包含在 ISA 定义范畴内的微结构层面的内容。举个例子来说，PC、通用寄存器、内存、处理器所处特权等级属于指令能看到的，而处理器中每一级流水的状态则属于指令看不到的。

指令又是怎么“看”的呢？这个过程的严谨而全面的表述是非常抽象且冗长的，这里我们表述得稍微工程化一些，通过几个典型的例子来说明。

1) 如果一条指令有寄存器的源操作数，那么它在 CPU 流水线中要到译码级生成源操作数的时候才开始“看”到这个寄存器。

2) 如果是一条 load 指令，那么它在 CPU 流水线中直到访存级^①才开始“看”到内存这个地址。

3) 如果是一条特权指令，即它只能在 CPU 处于特权状态下才能执行，假设有关特权指令合法性的检查是在译码级进行的，那么这条指令直到译码级才能“看”到 CPU 的特权等级状态。

4) 假设某些地址空间只有特权态的程序才能访问，那么任何指令必须在取指阶段发起

① 也许读者会说执行级发出数据 RAM 读请求的时候才是最早“看”到该地址内存值的时机。如果 load 指令在访存级获得访存结果只来自于数据 RAM 的 Q 端输出，那么这种看法就是完全正确的。如果 load 指令在访存级还有可能获得 store 指令直接前递过来的数值，那么正文中的表述就更为合适。正文中给出的是一种在大多数情况下都正确的表述。

访存请求的时候就“看”到 CPU 的特权等级状态。

从这四个例子中，读者多少能体会到指令的“看”在一个 CPU 中是如何进行的。之所以需要关注这个过程，是因为仅从程序员的角度来看，每条指令对于处理器的状态更新都应该是原子的、瞬时完成的，但是程序在处理器中真实运行的时候，一条指令涉及的处理器状态的更新却是分布的、有延迟的。作为处理器的设计人员，我们需要用这个分布的、有延迟的真实处理器给软件人员构造出一个原子的、瞬时的抽象处理器。

7.2 MIPS 指令系统中与例外相关的功能定义

在了解了例外的一般性概念的基础之上，我们需要具体分析 MIPS 指令系统中与例外相关的功能定义，以形成我们的最终设计。

7.2.1 CP0 寄存器

前面提到，例外是一个软硬件协同的处理过程。在这个过程中，硬件逻辑电路和例外处理软件需要进行必要的信息交互。为了实现精确例外，这些交互的信息不能放在用户态程序可见的软件上下文中。（请读者思考一下为什么。）MIPS 指令系统中定义了一组独立的寄存器用于这类信息的交互。由于 MIPS 指令系统将这些与特权态相关的功能都定义在协处理器 0（Coprocessor 0，CP0）中，因此我们也把其中定义的寄存器称为 CP0 寄存器。

就本章涉及的例外种类来说，相关的 CP0 寄存器有 Cause、EPC、Status、BadVAddr。这些寄存器的详细定义可参考附录 C 的第 6 节。为了帮助读者掌握这部分知识，建议首先快速浏览一遍第 6 节中相关寄存器的内容，这一遍可以“不求甚解”，对几个寄存器有个初步印象即可。然后阅读附录 C 中 5.1.3 节熟悉处理器硬件响应例外的一般性处理过程，之后可以研究 5.1.4 节～5.1.9 节来了解每个例外判定和处理的流程，中断处理的相关内容在附录 C 的 5.2 节中。最后，在敲定设计方案时，遇到与 CP0 寄存器相关的内容，可查阅第 6 节的内容，这一遍要尽量做到不遗漏任何细节。

此外，对附录 C 中 CP0 寄存器域描述表格中的“读/写”这一栏做一下补充说明。这里的“读/写”属性是对软件程序而言的。具体到本章，是指 MFC0 和 MTC0 两条指令能否读和写。那么，在例外处理过程中，这些 CP0 寄存器域能否被硬件逻辑电路写是在哪里定义的呢？请参阅附录 C 的 5.1.3 节～5.1.9 节的相关描述。

7.2.2 例外产生条件的判定

这一节我们将只考虑地址错、整型溢出、系统调用、断点、保留指令和中断这六种例外。若是从 Cause 寄存器需要填入的 Excode 值来看，则地址错例外还可进一步分为读地址错例外（AdEL）和写错误例外（AdES）。这七种例外的判定条件请查看附录 C 的 5.1.4 节～5.1.9 节以及 5.2 节的内容。下面我们对其中一些关键内容加以说明。

1. CPU 内部判定接收到中断的过程

注意，请将本部分的内容与附录 C 的 5.2 节结合起来阅读。

根据 MIPS 指令系统规范的定义，每个 MIPS 处理器核有 6 个中断输入引脚。外部的设备或中断控制器可以将电平中断信号[⊖]接入这 6 个中断输入引脚上。处理器核内部 CP0 的 Cause 寄存器的 IP7 ~ IP2 这六位（RTL 上对应 6 个触发器）直接对中断输入引脚的信号采样，且只能通过采样中断输入信号来进行设置。Cause 寄存器还有 IP1 和 IP0 两位，只能通过软件的方式（MTC0 写入）进行设置。Cause 寄存器 IP7 ~ IP0 的每一位都有一个对应的使能位，也就是 Status 寄存器的 IM7 ~ IM0。此外，还有一个全局的中断使能位，也就是 Status 寄存器的 IE 位。CPU 内部认为，接收到中断的标志信号 has_int 定义为：

```
has_int = (Cause.IP[7:0] & Status.IM[7:0])!=8'h00) && Status.IE==1'b1 && Status.EXL==1'b0;
```

简单解释一下，就是外部的中断通过处理器核的引脚设置 Cause 寄存器中对应的中断状态位，当这个中断状态位自身对应的使能位有效且全局使能位也有效，那么 CPU 内部就认为接收到中断。显然，这里 CPU 并不在意到底接收到的是一个外部中断还是多个外部中断。当确实同时接收到多个中断时，后续的处理交给软件上的中断处理函数进行。

2. 核内定时中断的产生过程

定时中断经常用于操作系统的调度和计时功能的实现。MIPS 指令系统规范定义了一个在核内实现的定时中断源。该中断源的核心有两个 CP0 寄存器：Count 和 Compare。顾名思义，Count 寄存器是一个 32 位计数器，它每经过两个处理器时钟周期自增一，加到 32'hFFFFFFFF 时直接溢出至 32'h0，然后继续计数，其间硬件不做任何特殊处理。Compare 寄存器是一个 32 位软件可读写的寄存器。当 Count 和 Compare 寄存器的值相等时，将 Cause 寄存器的 TI 位置为 1。Cause 寄存器的 IP7 位除了对处理器核的第 6 个中断输入引脚的信号采样，也同时对 Cause 寄存器的 TI 位采样。换言之，可以认为 Cause 寄存器的 TI 位是核内定时中断的中断状态位。（请读者思考一下：为什么不用 Count==Compare 的判断逻辑结果作为 Cause 寄存器的 IP7 位的采样输入？）Cause 寄存器的 TI 位不能被 MTC0 指令修改，但是当软件通过 MTC0 指令写 Compare 寄存器的时候，硬件需要将 Cause 寄存器的 TI 位清 0。

那么如何用这套机制产生一个固定时间间隔的定时中断源呢？假设处理器的主频不变，我们根据所需的时间间隔和主频计算出所需的时间间隔对应于 2A 个时钟周期。那么初始时，将 Count 置为全 0，Compare 置为 A。每次定时中断来的时候，中断处理程序用 MFC0 读出 Compare 寄存器的值，加上 A，再用 MTC0 写回 Compare 就可以了。现在，读者应该能理解为什么要通过写 Compare 寄存器来清 Cause 的 TI 位了。

⊖ 所谓电平中断信号是指当中断信号置为有效后，将一直保持在有效的电平上，直至中断处理程序响应中断后显式地清除中断源，该中断信号才会撤离有效电平。

7.2.3 例外入口

完整的 MIPS 指令系统规范中存在多个例外入口。本书中为了简单起见，目前只定义一个例外入口，所有类型的例外在发生后都跳转到地址 0xBFC00380 处取指执行。

7.2.4 MFC0 和 MTC0 指令

MIPS 指令系统中并没有定义直接操作 CP0 寄存器的算术逻辑运算类指令，而是定义了 MFC0 和 MTC0 指令用于在通用寄存器和 CP0 寄存器之间交互数据。举例来说，假如例外处理程序想查询 Cause 寄存器中的 Excode 域以确定当前处理的是哪种例外，那么它需要先用 MFC0 指令将 Cause 寄存器的值取到某个通用寄存器中，然后将这个通用寄存器中的第 6.2 位提取出来，得到 Cause 寄存器 Excode 域的值。再例如，如果想将 Status 寄存器的 IE 位置 0 以屏蔽所有的中断，该如何操作呢？由于此时只是修改 Status 寄存器的一部分，因此软件需要先用 MFC0 将 Status 寄存器的值取到某个通用寄存器，然后通过逻辑位与操作将这个通用寄存器的第 0 位置 0 而保持其他位不变，最后用 MTC0 将这个通用寄存器的值写回 Status 寄存器中。

7.2.5 ERET 指令

7.1 节中提到了在例外处理的结束阶段，例外处理程序要完成两个操作：一是回到例外出现的位置，二是恢复出现例外时的特权等级。这两个操作需要同步完成，即它们最好通过一条指令完成。在 MIPS 指令系统规范中，定义了 ERET 指令来同时完成这两个操作。ERET 指令一方面将 CP0 的 EPC 寄存器中存放的例外指令 PC 作为目标地址跳转过去，同时将 Status.EXL 清 0。将 Status.EXL 位清 0 是为了将处理器的特权等级恢复到出现例外的时刻。若你对此感到困惑，请回顾一下附录 C 中的描述，首先看看 5.1.3 节在开始响应例外阶段，处理器硬件对 Status 寄存器的 EXL 位做了什么操作，再看看 6.5 节 Status 寄存器的 EXL 位的概念描述是什么。

7.3 流水线 CPU 实现例外和中断的设计要点

7.3.1 例外检测逻辑

通过前面的基本概念梳理，我们已经知道例外发生条件的检测是由硬件完成的。因此，我们先来考虑地址错、整型溢出、系统调用、断点、保留指令和中断这几类例外的检测逻辑的实现。

1. 地址错例外

根据附录 C 中 5.1.5 节的描述，地址错例外的检测逻辑与取指和访存两部分相关。我们可以在取指级对取指所用的 PC 的最低两位进行判断，如果不是 2'b00 的话，说明取指出现了地址错例外 (AdEL)。同样，我们可以在访存级对访存操作类型以及访存地址的低位进行

判断。如果是 LW 操作且地址最低两位不等于 2'b00，或者是 LH、LHU 操作且地址最低位不等于 1'b0，那么意味着出现了地址错例外 (AdEL)；如果是 SW 操作且地址最低两位不等于 2'b00，或者是 SH 操作且地址最低位不等于 1'b0，那么表示出现了地址错例外 (AdES)。

2. 整型溢出例外

ADD、ADDI 和 SUB 这三条指令如果在计算过程中发生溢出，则发生整型溢出例外。附录 D 关于这三条指令的定义中给出了一种判断溢出例外的方法，不过需要将 ALU 中加法器改为 33 位数据宽。如果要延续之前给出的参考代码中的 32 位加法器，那么可以根据人判断溢出的思路来生成溢出信号，即对于加法，正数加正数得到负数，或者负数加负数得到正数；对于减法，正数减负数得到负数，或者负数减正数得到正数。

3. 系统调用例外

执行 SYSCALL 指令就会触发系统调用例外，所以一旦在译码级识别出 SYSCALL 指令，就可以认为发生了系统调用例外。

4. 断点例外

执行 BREAK 指令就会触发断点例外，所以一旦在译码级识别出 BREAK 指令，就可以认为发生了断点例外。

5. 保留指令例外

如果译码时发现指令码不是一条指令系统规范中定义的指令，那么就认为发生了保留指令例外。

6. 中断例外

有关中断例外以及定时中断的判断条件均在 MIPS 指令系统规范中有明确定义，7.2.1 节和 7.2.2 节也对这部分内容做了解释。这部分逻辑主要包括 Cause、Status、Count、Compare 这四个 CP0 寄存器的相关域，以及配套的 Count 的加 1 运算逻辑、Count 和 Compare 值是否相等的比较运算逻辑，以及中断是否发生的判断逻辑。这部分功能在指令系统规范中的定义比较具体，基本上可以对应规范直接转换出逻辑实现。

但是我们发现，前面所有的例外产生是指令（或程序）自身的属性造成的，而中断是由外部事件触发的，它与指令之间并无直接对应关系。既然我们将中断视作一种特殊的例外，那就意味着我们希望通过一套例外的处理框架，同时处理非中断和中断这两种属性的例外。我们采取的设计方法是，将异步的中断事件动态地标记在某一条指令上，被标记的指令随后就被赋予了中断这种例外，那么随后的处理就和其他例外非常相似了。在五级流水线 CPU 中，同一时刻可能会有多条指令，将中断标记在哪一条指令上合适呢？理论上讲，可以选出任一级流水线上的指令并标记上中断例外。出于精确例外实现开销和中断响应延迟两方面的权衡，我们通常将中断标记在译码级的指令上。但要注意的是，这不是唯一的设计方案。

细心的读者可能会问，既然 MIPS 只能接收电平中断输入，CPU 内部生成的是否有中断

的信号也会维持很多拍，那么会不会将很多指令都标记上中断例外呢？这是否会导致一些问题？答案是不会出现问题。首先，对于被中断例外打断的程序段，即使出现了多条指令被标记上中断例外的情况，根据接下来会讲到的精确例外实现的需要，只有程序序在最前面的那条指令才能报出中断例外，而程序序在它后面的那些指令会被取消，不会出现一个中断被多次报出的情况。至于硬件报出中断例外，进入中断例外处理程序之后，由于此时 Status 寄存器的 EXL 位是 1，因此 CPU 内部不会看到有中断发生。是不是此刻才真正理解了本章前面提到的生成中断发生信号的时候除了看各种中断使能位之外，还要看 Status 寄存器的 EXL 位的原因？

7.3.2 精确例外的实现

我们从指令系统规范的定义中知道，例外发生之后，处理器硬件需要设置一些 CP0 寄存器、进入最高特权等级并跳转到相应的例外入口。同时，我们通过 7.3.1 节的分析也了解到不同类型的例外可以发生在 CPU 的不同流水级。那么是不是一旦发生例外，处理器就要立即修改 CP0 寄存器的动作呢？如果我们这样处理，那么这些 CP0 寄存器、PC 因例外而更新的来源就会有多个，而同一时刻我们又只能选择一个来源对其进行更新，于是我们又会碰到如何选择的问题……事实上，为了实现精确例外，我们发现并不需要一发生例外就着急地修改 CP0 寄存器和 PC。

回顾一下 7.1.2 节中关于精确例外的分析，我们发现，发生例外时仅需要考虑如何处理那些在流水线中的指令。因为很显然，对于那些程序序在发生例外指令之前的指令，如果它们都已经执行完毕退出流水线，那么必然都已经产生了执行效果；而那些程序序在发生例外指令之后且还没有取进流水线的指令，等到它们真正取进流水线的时候，一定是在例外处理返回之后了。这些指令自然都遵循精确例外的语义。

那么发生例外时，那些已经在流水线中的指令该如何处理呢？具体的方法有很多种，这里介绍一种常用的设计思路：例外发生的判断逻辑分布在各流水级，靠近与之相关的数据通路；发现例外后将例外信息附着在指令上沿流水线一路携带下去，直至写回级才真正报出例外，此时才会根据所携带的例外信息更新 CP0 寄存器；写回指令报出例外的同时，清空所有流水级缓存的状态，并将下一拍的 PC 置为例外入口地址。当然，报出例外的流水级不一定要设定为最后一级（写回级），设定在执行级或访存级也可以，这样还会减少设计负担。但是要注意：选择报出例外的流水级的原则是，在该级之后的流水级不能产生新的例外（比如报出例外的流水级设定为执行级，则要求访存级和写回级不能产生或标记上新的例外），否则就违反了精确例外的要求。

简单分析一下上面这种做法的合理性。发生例外的指令到达写回级的时候，程序序在它前面的指令都已经退出流水线了，所以这些指令的执行效果都已经产生。写回级指令报例外时清空所有流水线，意味着那些已经进入 CPU 流水线、在例外指令之后（含例外指令）的指令都不会产生执行效果。例外入口地址最早是在报例外的下一拍才能进入 CPU 流水线，所

以它“看”到的处理器状态都是更新完成的状态。

上面的描述中，之所以给“都不会”三个字加上了下划线，是因为这样的特性并不是显然可得的。虽然所有对于通用寄存器的写是放在写回级进行的，但是 HI/LO 寄存器的更新是在执行级就完成了，store 类指令对于数据 RAM 的写命令也是在执行级就发出了。因此，如果当前拍写回级有指令要报例外而访存级是一条更新 HI/LO 的指令或 store 指令，则上一拍若没有在执行级对这些指令做任何处理，那么在报出例外的时候，HI/LO 寄存器或内存就已经被例外之后的指令修改了，这就违反了精确例外。那么该如何处理呢？现阶段一种简单有效的方式是：更新 HI/LO 的指令或 store 指令，若想在执行级发出写命令，那么需要检查当前访存级和写回级上是否存在已标记为例外或可能标记为例外的指令，也要检查自己有没有产生例外或标记例外。庆幸的是，就目前支持的指令和例外类型来说，指令在访存级和写回级不会再判断出新的例外了。也就是说，位于执行级的 store 指令只需要检查当前访存级和写回级上有没有已标记为例外的指令就可以了，当然，它也要在执行级检查自己有没有被标记上例外。这种情况下，例外信息都保存在流水线缓存触发器中，所以不会对数据 RAM 的写使能信号的时序造成特别大的影响。在真正实用的处理器中，store 指令不会在执行级就真的发出可修改内存的命令，而是要等到 store 指令从写回级执行完毕之后才真正发出修改内存的动作。这套功能通常需要 store buffer 或 store queue 这样的结构来支持。

7.3.3 CP0 寄存器

前一节介绍了 CPU 中实现例外的设计思路，其中有一个环节涉及报例外时对 CP0 寄存器的修改。由于几乎所有教材都没有介绍 CP0 寄存器如何实现，因此我们在这里介绍一下相关知识。

初学者在实现 CP0 寄存器时通常会有一种无从下手的感觉。我们认为，最主要的原因是 MIPS 指令系统规范中涉及 CP0 寄存器的内容被分散在不同章节，设计者首先要对这些知识点有全面的认识，才能把设计方案考虑周全。因此，请各位读者务必按照 7.2.1 节建议的学习过程来学习指令系统规范中与 CP0 寄存器相关的知识。

1. CP0 寄存器读、写来源梳理

有了这个认知基础，我们接下来着眼于 CP0 寄存器的各个域对这些知识点进行归纳梳理。记住，应该以 CP0 寄存器中的域为单位，而不要以一个 CP0 寄存器整体为单位。

(1) Status 寄存器的 BEV 域

从附录 D 的 6.5 节关于 Status 寄存器域的描述可知，BEV 这个域是软件只读不可写的^①，而且其中也没有硬件对其修改的定义，所以这个域本质上是一个常值。也就是说，可以不把它实现为触发器，而只将它看作一根具有固定值的线，即可以写成：

^① 这是与 MIPS 标准规范不一致的地方，目的是为了简化例外入口的生成逻辑，减轻读者的负担。在标准的 MIPS 规范中，Status 寄存器的 BEV 域是软件可写的。


```
wire          c0_status_bev;
assign c0_status_bev = 1'b1;
```

也可以把它写成:

```
reg          c0_status_bev;
always @(posedge clock) begin
    if (reset)
        c0_status_bev <= 1'b1;
end
```

这两种表述最终的效果是一样的。

BEV 域可以被软件读取, 则意味着 c0_status_bev 信号需要接入 MFC0 指令读 CP0 的数据通路中。

(2) Status 寄存器的 IM7 ~ IM0 域

Status 寄存器的 IM7 ~ IM0 域是软件可读写的, 意味着一方面这 8 个信号需要接入 MFC0 指令读 CP0 的数据通路中, 另一方面 MTC0 指令需要写这个寄存器。除了 MTC0 指令, 没有其他写的来源。对于 MTC0 指令, 我们生成一个它写 CP0 寄存器的写使能信号 mtc0_we, 代码如下:

```
wire          mtc0_we;
assign mtc0_we = wb_valid && op_mtc0 && !wb_ex;
```

在上面的代码中, MTC0 指令写 CP0 寄存器的操作是放在写回级进行的。这并不是唯一正确的设计方式, 只是维护精确例外的工作可以更简单一些。

我们还假设代码中 MTC0 指令写 CP0 寄存器的地址是 c0_addr[7:0], 写入数据是 c0_wdata[31:0], 那么 IM7 ~ IM0 域的代码可以写成:

```
reg [ 7:0] c0_status_im;
always @(posedge clock) begin
    if (mtc0_we && c0_addr==`CR_STATUS)
        c0_status_im <= c0_wdata[15:8];
end
```

上面的代码没有复位情况下的赋值, 这是因为指令系统规范中定义复位值是“无”, 表示硬件复位撤销后, 这个域的值是不确定的(其实, 也可以在 RTL 里给它一个明确的复位值)。软件需要知道这个约定, 并确保在使用 Status 寄存器的 IM7 ~ IM0 域的值之前将其置为确定的值。

IM7 ~ IM0 域是软件可读的, 意味着 c0_status_im[7:0] 需要接入 MFC0 指令读 CP0 的数据通路中。除了 MFC0, IM7 ~ IM0 域的值还参与处理器内部中断发生信号的生成, 意味着 c0_status_im[7:0] 还将被接入中断发生信号的生成逻辑中。

(3) Status 寄存器的 EXL 域

我们先来看看对于 Status 寄存器 EXL 域的写操作。在下面四种情况下会发生写操作:

- 1) 复位为 0。
- 2) MTC0 指令写 Status 寄存器。
- 3) 遇到 ERET 指令。
- 4) 任何指令报例外的时候。

我们假设 ERET 指令修改 EXL 域的使能信号是 `eret_flush`，写回级指令报例外的信号是 `wb_ex`，那么 Status 寄存器的 EXL 域维护的代码如下：

```
reg          c0_status_exl;
always @(posedge clock) begin
    if (reset)
        c0_status_exl <= 1'b0;
    else if (wb_ex)
        c0_status_exl <= 1'b1;
    else if (eret_flush)
        c0_status_exl <= 1'b0;
    else if (mtc0_we && c0_addr==`CR_STATUS)
        c0_status_exl <= c0_wdata[1];
end
```

在上面的代码中，为什么 ERET 指令修改 EXL 域的使能信号中加入了“`_flush`”这个成分？这是清空流水线的意思吗？答案是肯定的。那么为什么要清空流水线呢？我们将在 7.3.4 节的讨论中给出最终答案。

分析完了写再来分析读。EXL 域是软件可读的，这意味着 `c0_status_exl` 需要接入 MFC0 指令读 CP0 的数据通路中。除了 MFC0，EXL 域的值还参与处理器内部中断发生信号的生成，这意味着 `c0_status_exl` 还将被接入中断发生信号的生成逻辑中。另外，Status 寄存器的 EXL 域为 1 意味着处理器处于例外状态，它具有与核心态同等的特权等级，那么 CPU 中一切需要看特权等级的地方都要使用 `c0_status_exl`。最后这一点初学者经常会忘记，大家一定要牢记。

(4) Status 寄存器的 IE 域

与 Status 寄存器的 EXL 域相比，Status 寄存器的 IE 域的写来源就简单多了，只有下面两个：

- 1) 复位为 0。
- 2) MTC0 指令写 Status 寄存器。

因此，它的维护逻辑代码如下：

```
reg          c0_status_ie;
always @(posedge clock) begin
    if (reset)
        c0_status_ie <= 1'b0;
    else if (mtc0_we && c0_addr==`CR_STATUS)
        c0_status_ie <= c0_wdata[0];
end
```


IE 域的读也比较简单。它是软件可读的，这意味着 `c0_status_ie` 需要接入 MFC0 指令读 CP0 的数据通路中。除了 MFC0，IE 域的值还参与处理器内部中断发生信号的生成，这意味着 `c0_status_ie` 还将被接入中断发生信号的生成逻辑中。

(5) Cause 寄存器的 BD 域

Cause 寄存器的 BD 域是软件不可写的，只能在复位和指令报例外时更新。请注意，BD 域仅在 Status 寄存器 EXL 域不为 1 的时候才更新。假设流水线中每一级都新增一比特来标识指令是不是在转移指令延迟槽中，其对应到写回级的信号为 `wb_bd`，那么 BD 的维护逻辑如下：

```
reg          c0_cause_bd;
always @(posedge clock) begin
    if (reset)
        c0_cause_bd <= 1'b0;
    else if (wb_ex && !c0_status_exl)
        c0_cause_bd <= wb_bd;
end
```

Cause 寄存器的 BD 域是软件可读的，这意味着 `c0_cause_bd` 需要接入 MFC0 指令读 CP0 的数据通路中。

(6) Cause 寄存器的 TI 域

Cause 寄存器的 TI 域是软件不可写的，它有 3 个更新源：

- 1) 复位清 0。
- 2) Count 寄存器和 Compare 寄存器相等时置 1。
- 3) MTC0 指令写 Compare 寄存器时清 0。

我们假设 Count 寄存器和 Compare 寄存器相等比较结果信号为 `count_eq_compare`，那么 TI 域的维护逻辑如下：

```
reg          c0_cause_ti;
always @(posedge clock) begin
    if (reset)
        c0_cause_ti <= 1'b0;
    else if (mtc0_we && c0_addr==`CR_COMPARE)
        c0_cause_ti <= 1'b0;
    else if (count_eq_compare)
        c0_cause_ti <= 1'b1;
end
```

“if (mtc0_we && c0_addr==`CR_COMPARE)”和“if (count_eq_compare)”两个分支的条件从理论上讲是有可能同时发生的。我们推荐采用示例中的优先级，其基于的假设是：至少保证行为一定符合软件人员的预期，即他（她）们希望用 MTC0 写 Compare 寄存器就一定能清掉 Cause 寄存器的 TI 域。至于在这个过程中有没有可能 Count 寄存器恰好再一次等于 Compare 寄存器，就要由软件人员自己考虑和处理了。

Cause 寄存器的 TI 域是软件可读的, 这意味着 c0_cause_ti 需要接入 MFC0 指令读 CP0 的数据通路中。除此之外, TI 域会被 Cause 寄存器的 IP7 域采样。

(7) Cause 寄存器的 IP7 ~ IP2 域

首先请注意, 这里只分析 Cause 寄存器的 IP7 ~ IP2 这 6 比特, 它们也是软件不可写的。IP7 ~ IP2 的更新来源只有两个: 复位和采样中断输入信号。假设处理器核顶层的 6 个中断输入信号为 ext_int_in[5:0], 那么 IP7 ~ IP2 的维护逻辑示意如下:

```
reg [ 7:0] c0_cause_ip;
always @(posedge clock) begin
    if (reset)
        c0_cause_ip[7:2] <= 6'b0;
    else begin
        c0_cause_ip[7] <= ext_int_in[5] | c0_cause_ti;
        c0_cause_ip[6:2] <= ext_int_in[4:0];
    end
end
```

Cause 寄存器的 IP7 ~ IP2 域是软件可读的, 这意味着 c0_cause_ip[7:2] 需要接入 MFC0 指令读 CP0 的数据通路中。除了 MFC0, Cause 寄存器的 IP7 ~ IP0 域的值还参与处理器内部中断发生信号的生成, 这意味着 c0_cause_ip[7:2] 还将被接入到中断发生信号的生成逻辑中。

(8) Cause 寄存器的 IP1 和 IP0 域

Cause 寄存器的 IP1 和 IP0 域与 IP7 ~ IP2 域的区别在于, 前者只能通过软件来更新, 其维护逻辑的示意也很直观:

```
reg [ 7:0] c0_cause_ip;
always @(posedge clock) begin
    if (reset)
        c0_cause_ip[1:0] <= 2'b0;
    else if (mtc0_we && c0_addr == `CR_CAUSE)
        c0_cause_ip[1:0] <= c0_wdata[9:8];
end
```

Cause 寄存器的 IP1 和 IP0 域的读出与 IP7 ~ IP2 域是一样的, 不再赘述。

(9) Cause 寄存器的 Excode 域

Cause 寄存器的 Excode 域是软件不可写的, 它仅在指令报例外的时候填入例外编码。我们假设存在写回级的信号 wb_excode[4:0], 对应于写回级所报例外的编码。那么, Excode 域维护的代码示意如下:

```
reg [ 4:0] c0_cause_excode;
always @(posedge clock) begin
    if (reset)
        c0_cause_excode <= 5'b0;
    else if (wb_ex)
        c0_cause_excode <= wb_excode;
end
```


Cause 寄存器的 Excode 域是软件可读的，这意味着 c0_cause_excode 需要接入 MFC0 指令读 CP0 的数据通路中。

(10) EPC 寄存器

EPC 寄存器整体只有一个域。其更新来源有两个：一是 MTC0 指令可以写，二是报例外时硬件写入例外返回地址。对于第二个来源，一定要仔细看附录 C 的 6.7 节中的描述，有两点容易疏漏：一是 Status 寄存器的 EXL 域等于 1 时，即使报例外也不更新 EPC 寄存器；二是报例外指令如果是转移延迟槽指令，那么记录的不是报例外指令的 PC，而是该延迟槽对应的转移指令的 PC。指令系统规范之所以这么定义是有用意的，前一点是为了应对例外嵌套这种特殊的情况，后一点是确保用 ERET 指令从例外处理程序返回后源程序还能正确执行^①。我们假设写回级指令的 PC 存放在 wb_pc 中，那么 EPC 寄存器的维护代码示意如下：

```
reg [31:0] c0_epc;
always @(posedge clock) begin
    if (wb_ex && !c0_status_exl)
        c0_epc <= wb_bd ? wb_pc - 3'h4 : wb_pc;
    else if (mtc0_we && c0_addr == `CR_EPC)
        c0_epc <= c0_wdata;
end
```

EPC 寄存器是软件可读的，这意味着 c0_epc 需要接入 MFC0 指令读 CP0 的数据通路中。EPC 寄存器的值在执行 ERET 指令时将用于更新 PC，所以 c0_epc 还需要传递到取指部分的 GenNextPC 逻辑。

这里请各位读者考虑一个问题：为什么 EPC 寄存器要允许软件直接修改？给大家一个提示，请设想一个场景，如果发生系统调用例外（syscall 例外）的时候，例外处理完以后能直接返回最初硬件设置在 EPC 寄存器中的地址吗？

(11) BadVAddr 寄存器

BadVAddr 寄存器整体只有一个域，其更新来源只有一个，即报地址错例外的时候将导致出错的虚地址写入该寄存器。这里要注意以下几点：首先记录的是虚地址；其次如果是取指检查出的地址错例外，出错的虚地址就是 PC；最后，如果导致地址出错的原因是地址非对齐，那 BadVAddr 的低两位要严格地更新为非对齐地址的低两位。假定我们把这些报例外时需要填入 CP0 寄存器的信息沿流水线逐级传递下去，用来指导写回级统一更新 CP0 寄存器，那么千万不要为了省几个触发器而把各级流水线缓存中保存的 PC 的最低两位强行置为 0。假设我们一路携带至写回级的导致出错的虚地址存放在 wb_badvaddr 中，那么 BadVAddr 寄存器的维护逻辑示意如下：

① 提示一下：如果 ERET 返回到转移延迟槽指令继续执行，那么处理器内部已经没有该延迟槽对应的转移指令，请问该情况下处理器如何知晓该延迟槽对应的转移指令是跳转还是不跳转？如果不能正确知晓，请问重新执行的程序如何能正确执行？

```

reg [31:0] c0_badvaddr;
always @(posedge clock) begin
    if (wb_ex && wb_excode==`EX_ADEL)
        c0_badvaddr <= wb_badvaddr;
end

```

BadVAddr 寄存器是软件可读的，这意味着 c0_badvaddr 需要接入 MFC0 指令读 CP0 的数据通路中。

(12) Count 寄存器

Count 寄存器整体只有一个域，其更新来源有两个：MTC0 指令和硬件计数器。它的维护逻辑示意如下：

```

reg          tick;
reg [31:0] c0_count;
always @(posedge clock) begin
    if (reset) tick <= 1'b0;
    else      tick <= ~tick;

    if (mtc0_we && c0_addr==`CR_COUNT)
        c0_count <= c0_wdata;
    else if (tick)
        c0_count <= c0_count + 1'b1;
end

```

上面的代码中引入 tick 触发器是为了达到指令规范中定义的每两个周期加 1 的效果。

Count 寄存器是软件可读的，这意味着 c0_count 需要接入 MFC0 指令读 CP0 的数据通路中。

(13) Compare 寄存器

Compare 寄存器整体只有一个域。其自身的功能极其简单，包括被 MTC0 指令写以及被 MFC0 指令读。我们就不再多做介绍了。

2. CP0 寄存器的代码组织风格

我们建议大家把 CP0 寄存器封装成一个模块，前面所述的 CP0 寄存器各个域的读、写信号都作为这个模块的接口。如果大家觉得进行实例化的时候连接很多端口比较麻烦，也可以考虑将 CP0 寄存器放在更新发生的那一级流水的模块中。目前来看，例外和 MTC0 写都发生在写回级，所以将其放在写回级模块也可以。但是，所有需要读出的信息都要从写回级模块引出来，这些代码是省不掉的。

7.3.4 CP0 冲突

我们介绍流水线 CPU 设计的时候，曾经专门分析过围绕通用寄存器的数据相关以及由此而产生的数据相关冲突。这里将要分析的 CP0 冲突 (CP0 hazard) 与之类似，它是由围绕 CP0 寄存器的“写后读”相关引起的，因 CPU 采用了多级流水线结构设计而显现出来。

典型的 CP0 写后读相关是 MTC0 指令写一个 CP0 寄存器，然后用 MFC0 指令读这个 CP0 寄存器。为了解决这个相关所引起的冲突，最简单、有效的方式是将 MFC0 和 MTC0 指令读、写 CP0 寄存器放到同一级流水线处理。这种解决方案和前面处理 HI/LO 寄存器读写的思路是一样的。这种方案也会造成一定的性能损失。例如，我们目前建议将 MTC0 放到写回级处理，那么 MFC0 也是在写回级读 CP0 寄存器，因此 MFC0 指令和后续需要其结果的指令之间就存在 2 拍的执行延迟，也就是说，需要其结果的指令必须将自身阻塞在译码级直至 MFC0 指令到达写回级才能继续执行。虽然我们也可以在 MTC0 和 MFC0 指令之间建立前递路径，但是 MFC0 和 MTC0 本身就是特权指令，只存在于内核之类的特权软件中，而且 MTC0 写入一个 CP0 寄存器立即就用 MFC0 将其读出来的情况也不多见。这种小概率场景的性能优化所带来的整体性能提升效果微乎其微，投入产出比太低，因此我们并不考虑。

之所以要专门用一节来介绍 CP0 冲突，是因为它不仅由 MTC0 → MFC0 这一类相关引起。通过 7.3.3 节的分析，我们已知 CP0 寄存器的各个域有不同的读者、写者。这些存在写后读相关的写者和读者只要不是在同一级流水级，就会遇到 CP0 冲突。由于我们在本书中刻意删除了指令系统中与特权等级维护、检查相关的内容，因此碰到的 CP0 冲突情况并不多，目前只有如下两种：

	写者	相关对象	读者
1	MTC0	Status.EXL, Status.IE, Status.IM, Cause.IP1/0, Compare	译码级指令（标记中断）
2	ERET	Status.EXL; EPC	译码级指令（标记中断）； 取指 PC

上面的第 1 种情况用硬件处理起来略显复杂（因为主要涉及中断处理程序的代码，所以我们通过对软件加以约束来规避冲突）。对于第 2 种情况，我们采用硬件方式解决。具体方案是 ERET 指令直到写回级才清 Status 寄存器的 EXL 域，与此同时，清空流水线并更新取指 PC。这就是前面提到的 `eret_flush` 信号的由来。对于流水级缓存来说，这个信号与例外信号 `wb_ex` 的作用是一样的，不同之处在于它不是一个软件可见的例外。其实，也可以通过类似的方式来处理第 1 种情况中 MTC0 指令写 CP0 寄存器所带来的 CP0 冲突，感兴趣的读者可以自行尝试一下。

7.4 任务与实践

完成本章的学习后，读者应该能够完成以下 2 个实践任务：

- 1) 添加 `syscall` 例外支持，实践资源见 `lab8.zip`。
- 2) 添加其他例外与中断支持，实践资源见 `lab9.zip`。

为完成以上实践任务，需要参考的文档包括但不限于：

- 1) 本章内容。
- 2) 附录 C。

7.4.1 实践任务一：添加 syscall 例外支持

本实践任务要求在 lab7 完成的 CPU 基础上，继续完成以下工作：

- 1) 为 CPU 增加 MTC0、MFC0、ERET 指令。
- 2) 为 CPU 增加 CP0 寄存器 Status、Cause、EPC。
- 3) 为 CPU 增加 SYSCALL 指令，也就是增加 syscall 例外支持。
- 4) 运行 lab8.zip 里的 func_lab8，要求成功通过仿真和上板验证。

本实践任务的硬件环境沿用 CPU_CDE，软件环境使用 lab8.zip。lab8.zip 只包含软件编译环境 (func_lab8)。func_lab8 和 func_lab7 相比，加入了 syscall 例外的测试内容。

请参考下列步骤完成本实践任务：

- 1) 请先完成第 6 章的学习。
- 2) 学习本章内容。
- 3) 准备好 lab7 的实验环境和 CPU_CDE，该环境也会作为 lab8 的实验环境。
- 4) 解压 lab8.zip，将 func_lab8/ 拷贝到 CPU_CDE/soft/ 目录里，与 func_lab7 同层次。
- 5) 打开 cpu132_gettrace 工程 (CPU_CDE/cpu132_gettrace/run_vivado/cpu132_gettrace/cpu132_gettrace.xpr)。
- 6) 对 cpu132_gettrace 工程中的 inst_ram 进行重新定制，此时选择加载 func_lab8 的 coe (CPU_CDE/soft/func_lab8/obj/inst_ram.coe)。
- 7) 运行 cpu132_gettrace 工程的仿真 (进入仿真界面后，直接点击 run all 等待仿真运行完成)，生成新的参考 Trace 文件 golden_trace.txt (CPU_CDE/cpu132_gettrace/golden_trace.txt)。注意，要等仿真运行完成，golden_trace.txt 才有完整的内容。
- 8) 打开 myCPU 工程 (CPU_CDE/mycpu_verify/run_vivado/mycpu_prj1/mycpu_prj1.xpr)。
- 9) 对 myCPU 工程中的 inst_ram 进行重新定制，此时选择加载 func_lab8 的 coe (CPU_CDE/soft/func_lab8/obj/inst_ram.coe)。
- 10) 运行 myCPU 工程的仿真 (进入仿真界面后，直接点击 run all)，开始调试。
- 11) myCPU 仿真通过后，综合实现后生成比特流文件，进行上板验证 (如果没有实验箱，请跳过这一步)。

7.4.2 实践任务二：添加其他例外支持

本实践任务要求在 lab8 完成的 CPU 基础上，继续完成以下工作：

- 1) 为 CPU 增加 Break、地址错、整数溢出、保留指令例外支持。
- 2) 为 CPU 增加 CP0 寄存器 Count、Compare、BadVAddr。
- 3) 为 CPU 增加时钟中断支持，时钟中断要求固定绑定在硬件中断 5 号上，也就是 CP0

寄存器 Cause 的 IP7 上。

- 4) 为 CPU 增加 6 个硬件中断支持, 编号为 0 ~ 5, 对应 CP0 寄存器 Cause 的 IP7 ~ IP2。
- 5) 为 CPU 增加 2 个软件中断支持, 对应 CP0 寄存器 Cause 的 IP1 ~ IP0。
- 6) 运行 lab9.zip 里的 func_lab9, 要求成功通过仿真和上板验证。
- 7) 推荐上板运行 lab9 的记忆游戏程序 (memory_game)。

本实践任务的硬件环境沿用 CPU_CDE, 软件环境使用 lab9.zip。lab9.zip 只有两个软件编译环境: func_lab9 和 memory_game。func_lab9 和 func_lab8 相比, 加入了前述的新增例外与中断的测试内容。

请参考下列步骤完成实践任务:

- 1) 学习本章内容。
- 2) 准备好 lab8 的实验环境和 CPU_CDE, 该环境会也会作为 lab9 的实验环境。
- 3) 解压 lab9.zip, 将 func_lab9/ 拷贝到 CPU_CDE/soft/ 目录里, 与 func_lab8 同层次。
- 4) 打开 cpu132_gettrace 工程 (CPU_CDE/cpu132_gettrace/run_vivado/cpu132_gettrace/cpu132_gettrace.xpr)。
- 5) 对 cpu132_gettrace 工程中的 inst_ram 进行重新定制, 此时选择加载 func_lab9 的 coe (CPU_CDE/soft/func_lab9/obj/inst_ram.coe)。
- 6) 运行 cpu132_gettrace 工程的仿真 (进入仿真界面后, 直接点击 run all 等待仿真运行完成), 生成新的参考 trace 文件 golden_trace.txt (CPU_CDE/cpu132_gettrace/golden_trace.txt)。注意, 要等仿真运行完成, golden_trace.txt 才有完整的内容。
- 7) 打开 myCPU 工程 (CPU_CDE/mycpu_verify/run_vivado/mycpu_prj1/mycpu_prj1.xpr)。
- 8) 对 myCPU 工程中的 inst_ram 进行重新定制, 此时选择加载 func_lab9 的 coe (CPU_CDE/soft/func_lab9/obj/inst_ram.coe)。
- 9) 运行 myCPU 工程的仿真 (进入仿真界面后, 直接点击 run all), 开始调试。
- 10) myCPU 仿真通过后, 综合实现后生成 bit 流文件, 进行上板验证 (如果没有实验箱, 请跳过这一步)。

在 func_lab9 功能测试通过后, 推荐上板运行记忆游戏 (lab9.zip 里的 memory_game)。记忆游戏不提供仿真运行环境, 如果没有实验箱, 请跳过该步。记忆游戏综合实现生成比特流文件的步骤如下:

- 1) 准备好 lab9 的实验环境和 CPU_CDE。
- 2) 将 lab9.zip 里的 memory_game 拷贝到 CPU_CDE/soft/ 目录里, 与 func_lab9 同层次。
- 3) 打开 myCPU 工程 (CPU_CDE/mycpu_verify/run_vivado/mycpu_prj1/mycpu_prj1.xpr)。
- 4) 对 myCPU 工程中的 inst_ram 进行重新定制, 此时选择加载 memory_game 的 coe (CPU_CDE/soft/memory_game/obj/inst_ram.coe)。
- 5) 进行综合、实现和生成比特流文件。
- 6) 将生成的比特流文件下载到开发板上, 进行记忆游戏的测试。

下载比特流文件到实验箱上后，记忆游戏的测试步骤如下：

- 1) 按实验箱上矩阵键盘的最下面一行的4个按键中的任一按键，开始游戏。
- 2) 单色LED灯最右侧4个灯会随机点亮，共点亮8次。
- 3) 努力回忆8次点亮顺序，使用矩阵键盘的最下面一行的4个按键复现8次点亮顺序。
- 4) 按8次后，数码管左侧会展示8（表示共8次点亮），右侧会展示你记忆正确的次数。
- 5) 如果8次都记忆对了，则数码管右侧会展示8，且双色LED灯会亮2个绿色；如果不是8次都对，双色LED灯会亮一红一绿。

如果发现记忆游戏运行不正确，可以按照以下步骤调试：

- 1) 确认 `func_lab9` 运行无错。
- 2) 可以尝试对记忆游戏进行仿真，但需要自己设计按键的激励，且需要看懂记忆游戏源码并修改 `delay` 函数。
- 3) 可以尝试使用逻辑分析仪进行在线调试，参考附录 D.1 节。
- 4) 反思自己的设计、代码规范、综合时序等。